

Onto the Network

The board becomes a web server — a sensor reading reaches your phone with no app and no cable. The ESP32's superpower, in twenty lines.

ESP32-007 · ~35 MIN · INTERMEDIATE

1. The one idea

The board **is** a web server. The ESP32 joins your WiFi and serves its own page, so a sensor reading shows up in any browser on the network — no app, no driver, no USB cable once it's running.

2. Why it matters

This is the ESP32's superpower and the reason it's the bridge to the phone and the web. The moment the board speaks HTTP, every browser on the network is a ready-made UI — no app to install, no platform to fight. It's the difference between a gadget tethered to your laptop and an instrument anyone can open on their phone.

3. Parts & wiring

No new hardware — the Course 3 bench: ESP32 + the BME280 on I2C. You'll also need your WiFi name and password, and a phone or laptop on the **same 2.4 GHz network**.

4. Predict (do this before uploading)

- When you load the page, where does the HTML come from — your laptop or the ESP32?
- What address will you type into the browser, and where does the board print it?
- Your home WiFi is 5 GHz. Will the ESP32 join it?

5. Build it — serve a page and a data feed

```
#include <WiFi.h>
#include <WebServer.h>
#include <Adafruit_BME280.h>

Adafruit_BME280 bme;
WebServer server(80);

void handleRoot() {
  server.send(200, "text/html",
    "<h2>ESP32 sensor</h2><p id=t> ... </p>"
    "<script>setInterval(async()=>{"
    "t.textContent=(await (await fetch('/data')).json()).temp+' C';"
    "},1000)</script>");
}

void handleData() {
  char j[48];
  snprintf(j, sizeof(j), "{\"temp\":%.1f}", bme.readTemperature());
  server.send(200, "application/json", j);
}

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);
  bme.begin(0x76);
  WiFi.begin("YOUR_SSID", "YOUR_PASS");
  while (WiFi.status() != WL_CONNECTED) delay(200);
  Serial.println(WiFi.localIP());           // ← open this in a browser
  server.on("/", handleRoot);
  server.on("/data", handleData);
  server.begin();
}

void loop() { server.handleClient(); }
```

The new idea: the ESP32 answers requests. `handleRoot` hands the browser a page; the page's script asks `/data` once a second for a fresh reading. Two endpoints, and the board is on the network.

6. Test against your prediction

Open the Serial Monitor, copy the IP it prints, and type it into your phone's browser. The page shows the temperature and updates every second — served straight off the board, no cable. Was the IP source what you predicted?

7. What goes wrong (pre-loaded debugging)

- **Never connects (stuck in the while loop)** → wrong SSID/password, or a **5 GHz** network — the ESP32 is 2.4 GHz only. The classic trap.
- **Connects but the page won't load** → your phone is on a different network/subnet, or the AP has client isolation on. Use the same 2.4 GHz network.
- **Page loads once then freezes** → something blocks `server.handleClient()` with a long `delay()`. Keep `loop()` quick.
- **Reading never changes** → you cached one value; re-read the sensor inside `handleData`.

8. Checkpoint

The board is a networked instrument now — it serves its own UI to any browser, no app and no cable. That single capability is what turns every sensor and guard you've built into something you can watch from across the room or across the house.

9. Push further

- Add `ESPmDNS` so you can browse to `esp32.local` instead of an IP.
- Run a SoftAP (`WiFi.softAP`) so the board makes its *own* network — no router needed in the field.
- This request/response page becomes a live *stream* in Course 8.